

What is Model Context Protocol (MCP)? How it simplifies AI integrations compared to APIs

March 7, 2025 · 7 min read



Norah Klintberg Sakal

AI Consultant & Developer



GUIDE

What is Model Context Protocol (MCP)?

Learn how MCP provides a standardized way to connect AI agent to tools and data sources

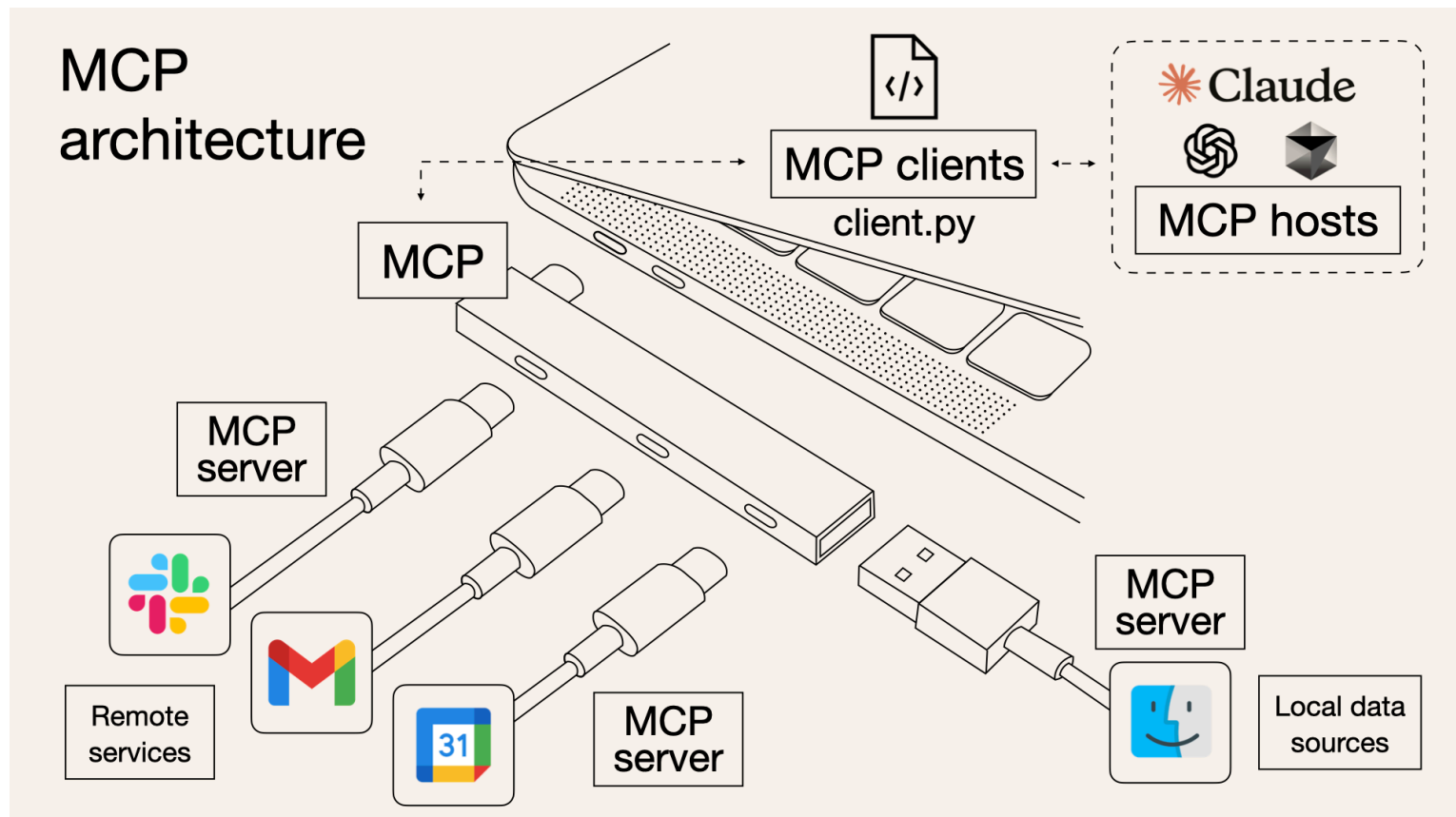
MCP (Model Context Protocol) is a new open protocol designed to standardize how applications provide context to Large Language Models (LLMs).

Think of MCP like a USB-C port but for AI agents: it offers a uniform method for connecting AI systems to various tools and data sources.

This post breaks down MCP, clearly explaining its value, architecture, and how it differs from traditional APIs.

What is MCP?

The Model Context Protocol (MCP) is a standardized protocol that connects AI agents to various external tools and data sources. Imagine it as a USB-C port - but for AI applications.



The **Model Context Protocol (MCP)** is a standardized protocol that connects AI agents to various external tools and data sources

Just as USB-C simplifies how you connect different devices to your computer, MCP simplifies how AI models interact with your data, tools, and services.

Why use MCP instead of traditional APIs?

Traditionally, connecting an AI system to external tools involves integrating multiple APIs. Each API integration means separate code, documentation, authentication methods, error handling, and maintenance.

Why traditional APIs are like having separate keys for every door

Metaphorically Speaking: APIs are like individual doors - each door has its own key and rules:

APIs: Every tool needs its own key

Traditional APIs require different authentication and integration for each service, like needing different keys for different locks



Traditional APIs require developers to write custom integrations for each service or data source

Who's behind MCP?

MCP (Model Context Protocol) started as a project by [Anthropic](#) ↗ to make it easier for AI models - like Claude - to interact with tools and data sources.

But it's not just an Anthropic thing anymore. MCP is open, and more companies and developers are jumping on board.

It's starting to look a lot like a new standard for AI-tool interactions.



TIP

Curious to dig deeper? The official MCP spec and ongoing development can be found at modelcontextprotocol.io ↗.

MCP vs. API: Quick comparison

Feature	MCP	Traditional API
Integration Effort	Single, standardized integration	Separate integration per API

Feature	MCP	Traditional API
Real-Time Communication	✔ Yes	✗ No
Dynamic Discovery	✔ Yes	✗ No
Scalability	Easy (plug-and-play)	Requires additional integrations
Security & Control	Consistent across tools	Varies by API

Key differences between MCP and traditional APIs:

- **Single protocol:** MCP acts as a standardized "connector," so integrating one MCP means potential access to multiple tools and services, not just one
- **Dynamic discovery:** MCP allows AI models to dynamically discover and interact with available tools without hard-coded knowledge of each integration
- **Two-way communication:** MCP supports persistent, real-time two-way communication - similar to WebSockets. The AI model can both retrieve information and trigger actions dynamically

 **WHY TWO-WAY COMMUNICATION?**

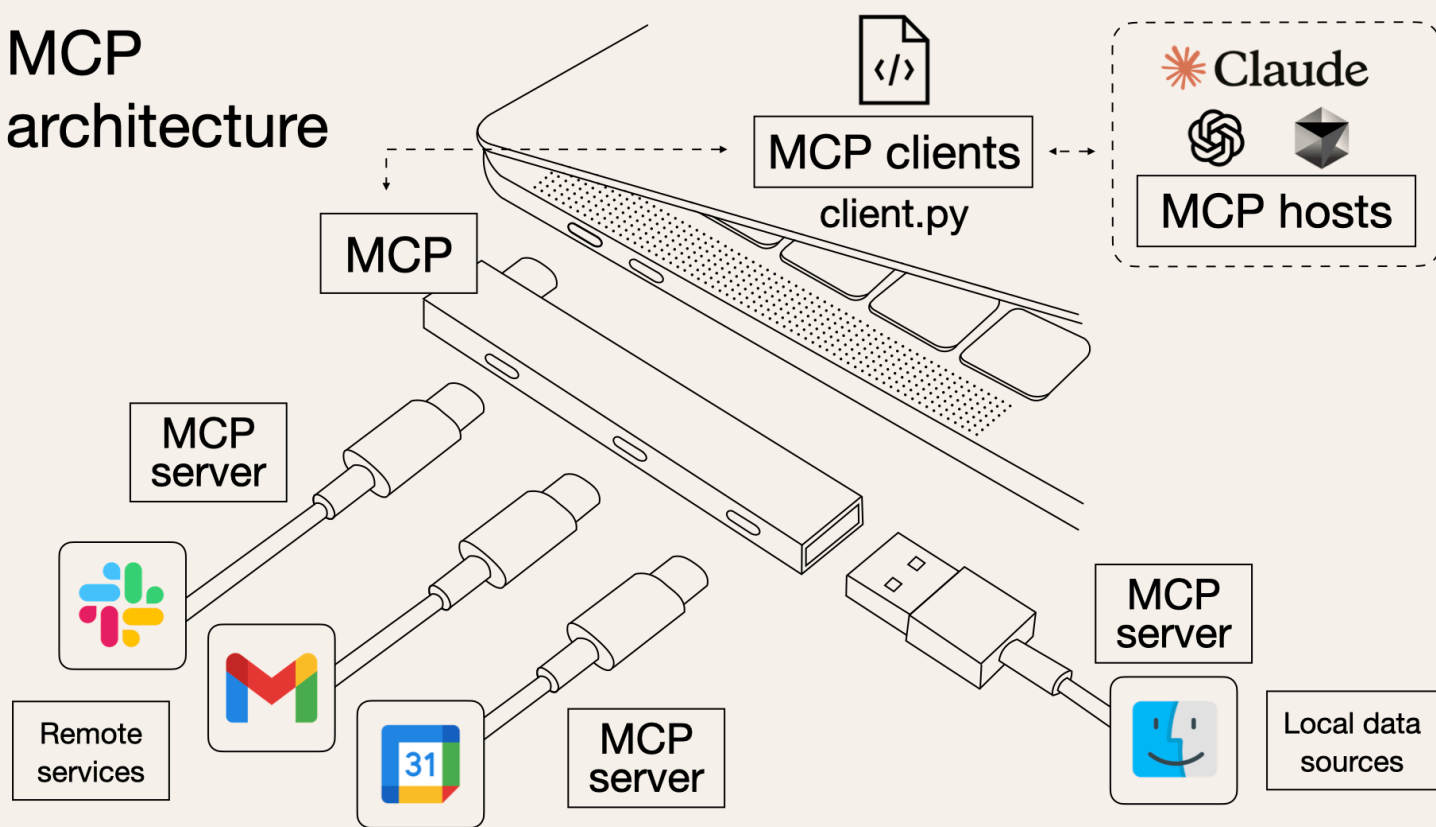
MCP provides real-time, two-way communication:

- **Pull data:** LLM queries servers for context → e.g. checking your **calendar**
- **Trigger actions:** LLM instructs servers to take actions → e.g. **rescheduling meetings, sending emails**

How MCP works: The architecture

MCP follows a simple client-server architecture:

MCP architecture



- **MCP Hosts:** These are applications (like Claude Desktop or AI-driven IDEs) needing access to external data or tools
- **MCP Clients:** They maintain dedicated, one-to-one connections with MCP servers
- **MCP Servers:** Lightweight servers exposing specific functionalities via MCP, connecting to local or remote data sources
- **Local Data Sources:** Files, databases, or services securely accessed by MCP servers
- **Remote Services:** External internet-based APIs or services accessed by MCP servers

Visualizing MCP as a bridge makes it clear: MCP doesn't handle heavy logic itself; it simply coordinates the flow of data and instructions between AI models and tools.



TIP

Just as USB-C simplifies how you connect different devices to your computer, MCP simplifies how AI models interact with your data, tools, and services

An MCP client in practice

In practice, an MCP client (e.g., a Python script in `client.py`) communicates with MCP servers that manage interactions with specific tools like **Gmail**, **Slack**, or **calendar apps**.

This standardization removes complexity, letting developers quickly enable sophisticated interactions.

MCP examples: When to use MCP?

Consider these scenarios:

1. Trip planning assistant

- **Using APIs:** You'd write separate code for Google Calendar, email, airline booking APIs, each with custom logic for authentication, context-passing, and error handling
- **Using MCP:** Your AI assistant smoothly checks your **calendar** for availability, **books flights**, and **emails confirmations** - all via MCP servers, no custom integrations per tool required

2. Advanced IDE (Intelligent Code Editor)

- **Using APIs:** You'd manually integrate your IDE with file systems, version control, package managers, and documentation
- **Using MCP:** Your IDE connects to these via a single MCP protocol, enabling richer context awareness and more powerful suggestions

3. Complex data analytics

- **Using APIs:** You manually manage connections with each database and data visualization tool
- **Using MCP:** Your AI analytics platform autonomously discovers and interacts with multiple databases, visualizations, and simulations through a unified MCP layer

Benefits of implementing MCP

- **Simplified development:** Write once, integrate multiple times without rewriting custom code for every integration
- **Flexibility:** Switch AI models or tools without complex reconfiguration
- **Real-time responsiveness:** MCP connections remain active, enabling real-time context updates and interactions
- **Security and compliance:** Built-in access controls and standardized security practices
- **Scalability:** Easily add new capabilities as your AI ecosystem grows—simply connect another MCP server

When are traditional APIs better?

If your use case demands precise, predictable interactions with strict limits, traditional APIs could be preferable. MCP provides broad, dynamic capabilities ideal for scenarios requiring flexibility and context-awareness but might be less suited for highly controlled, deterministic applications.

Stick with granular APIs when:

- Fine-grained control and highly-specific, restricted functionalities are needed
- You prefer tight coupling for performance optimization
- You want maximum predictability with minimal context autonomy

Getting started with MCP: High-level steps

MCP integration:

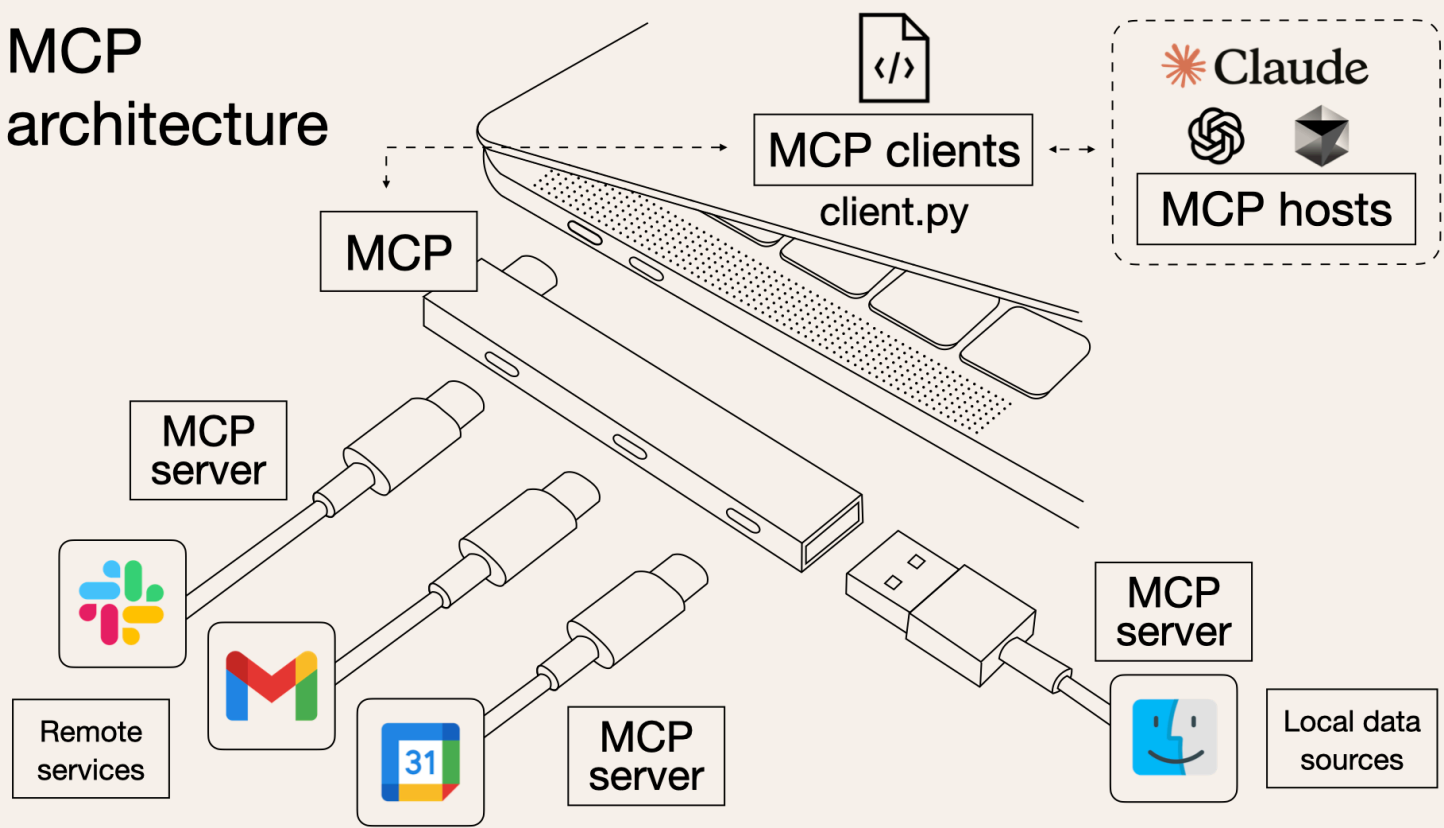
1. **Define capabilities:** Clearly outline what your MCP server will offer
2. **Implement MCP layer:** Adhere to the standardized MCP protocol specifications
3. **Choose transport:** Decide between local (stdio) or remote (Server-Sent Events/WebSockets)
4. **Create resources/tools:** Develop or connect the specific data sources and services your MCP will expose
5. **Set up clients:** Establish secure and stable connections between your MCP servers and clients

Summary

What is MCP?

- **MCP:** Unified interface for AI agents to dynamically interact with external data/tools
- **APIs:** Traditional methods, requiring individualized integrations and more manual oversight

MCP architecture



MCP provides a **unified** and **standardized** way to integrate AI agents and models with external data and tools

Conclusion

MCP provides a **unified** and **standardized** way to integrate AI agents and models with external data and tools. It's not just another API; it's a powerful connectivity framework enabling intelligent, dynamic, and context-rich AI applications.

💡 **NEED HELP IMPLEMENTING MCP OR EXPLORING AI INTEGRATIONS?**

Reach out for consulting: norah@braine.ai or schedule a [free brainstorming session](#)

Tags: [Model Context Protocol](#) [MCP](#) [AI Integrations](#) [API vs MCP](#) [Large Language Models](#)

[AI Architecture](#)

[AI Tooling](#)

[AI Development](#)

[Enterprise AI](#)

[AI Agents](#)